

The algebra of the *random approximate map* model

Alexander Towell
atowell@siue.edu

Abstract

We define the semantics of the *approximate map*, which is a *probabilistic* map that *approximates* another map with a false positive rate ε and false negative rate ω . We derive several properties that follow from these semantics, such as the expected *precision* in information retrieval. Following from this, we generalize to approximate relations and compositions. Finally, we demonstrate an application especially relevant to *Encrypted Search*, rank-ordered search, which generates *approximate result sets* with approximate values and show how the relational model may be used to support different ranking methods. Finally, we extend this information retrieval model to a fuzzy set-theoretic query model.

Contents

1	Introduction	1
2	Approximate maps	2
2.1	Rate-distortion	4
3	Random approximate maps	4
3.1	Approximate set-indicator function	5
3.2	Probability model	6
4	Higher-order random approximate maps	7
4.1	Algebra of relations	8
4.2	Algebra of sets	8
4.3	Composition	8
5	Predicate functions	9
5.1	Indicator functions (unary predicate functions)	9
5.2	Relations as n -ary predicates	10
5.2.1	Total orders	10
6	Discrete partial functions	10
6.1	0-ary functions (constants)	10
7	A first-order rate-distorted map	11
7.1	Maybe monad	13
7.2	Theoretical analysis	15
7.3	Entropy	16

1 Introduction

A *map*, also known as a *function*, associates elements in a domain to elements in a codomain. This association may only be partial over the domain, in which case it is a *partial function* and otherwise is a *total function*. A total function is commonly called just a *function*. A partial function $f: X \rightarrowtail Y$ may be thought of as a total function $f: X \mapsto \mathfrak{m}(Y)$.

Let $f: X \mapsto Y$ be a function. The concept of a Bernoulli map of f , denoted by $AF(f): X \mapsto Y$, models f with *error*, either do to rate distortion, noise, or ignorance, such that

$$P[AF(f)(x) \neq f(x) \mid x \in A] = \epsilon_A . \quad (1)$$

with two general categories of error:

1. Elements in the domain of definition may be undefined.
2. Undefined elements may be defined.

There are also random approximate value types.

The **Void** and **Unit** types have respectively 0 and 1 elements, therefore there are no approximations of these types. The most primitive type which has an approximation is the approximate Boolean type, in which under the *random approximate type model*, the element $\mathbf{true}_\epsilon^\tau$ is Bernoulli distributed that realizes **true** with probability τ and **false** with probability ϵ and similarly for $\mathbf{false}_\epsilon^\tau$.

Given **Void**, **Unit**, and $\mathbf{Bool}^\sigma$, we may algebraically compose them to generate other types. For instance, the product type $\mathbf{Bool}^\sigma \times \mathbf{Bool}^\sigma$ is a random approximate product of Booleans. By the product rule in probability,

$$P[\langle \mathbf{true}_{\epsilon_1}^{\tau_1}, \mathbf{true}_{\epsilon_2}^{\tau_2} \rangle = \langle \mathbf{true}, \mathbf{true} \rangle] = \tau_1 \tau_2 . \quad (2)$$

Under the random approximate Boolean model, the logical functions $\mathbf{and}: \mathbf{Bool} \times \mathbf{Bool} \mapsto \mathbf{Bool}$ and $\mathbf{not}: \mathbf{Bool} \mapsto \mathbf{Bool}$ are also random. In particular, $\mathbf{not}(\mathbf{true}_b^a)$ and $\mathbf{not}(\mathbf{false}_b^a)$ are respectively distributed as $\mathbf{false}_a^b$ and $\mathbf{true}_a^b$.

These results generate the random approximate Boolean algebra

$$\left(\mathbf{Bool}_{\epsilon_1}^{\tau_1} \times \mathbf{Bool}_{\epsilon_2}^{\tau_2}, \mathbf{and}, \mathbf{or}, \mathbf{not}, \langle \mathbf{true}_{\epsilon_1}^{\tau_1}, \mathbf{true}_{\epsilon_2}^{\tau_2} \rangle, \langle \mathbf{false}_{\epsilon_1}^{\tau_1}, \mathbf{false}_{\epsilon_2}^{\tau_2} \rangle \right) , \quad (3)$$

which only *approximately* obeys the axioms.

Any set U with operations set-intersection and set-complement may be used to form the Boolean algebra

$$\left(2^U, \cap, \cup, ^-, U, \emptyset \right) . \quad (4)$$

If $|2^U| = 4$, then it is *isomorphic* to the Boolean algebra given by eq. (3). The *approximate algebra of sets* over a universe U is thus given by

$$\left((\mathbf{Bool}_\epsilon^\tau)^n, \mathbf{and}, \mathbf{or}, \mathbf{not}, \left(\mathbf{true}_0^1 \right)^n, \left(\mathbf{false}_0^1 \right)^n \right) . \quad (5)$$

Functions of the type $X \mapsto Y$ where X and Y may be any algebraic type, e.g., $X = X_1 \times X_2$ for a *binary function*, has an algebraic notation Y^X .

A *random approximate exponential type* is a partial or total function, $f^\pm: X \rightarrow Y$, with a domain of definition $\text{def}(f_\tau^\varepsilon) = (\text{def}(f))_\tau^\varepsilon$ but whose range depends on the function. For example, if f is a bijection when restricted to its domain of definition, then $\text{range}(f_\tau^\varepsilon) = (\text{range}(f))_\tau^\varepsilon$.

The approximate map is an example of a class of *approximate relations* where the relation has a functional constraint. Other constraints include properties like *symmetric* or *reflexivity*. Some constraints, however, are difficult to implement under the random approximate type model. However, note that, more generally, the approximation error can be applied to *any* predicate.

The approximate map is closely related to the *approximate set*[?] and *approximate relation*[?]. Approximate maps are one of the key components in an algebraic data type corresponding to the *exponential type*.

Bernoulli distributed, such that anything that composes them, e.g., a *random approximate set* over a universe of n elements may be modeled as the product of n random approximate Booleans.

In section 2, we precisely define the approximate map. In the subsequent sections, we explore properties of approximate maps, such as its *mean average precision* with respect to particular scoring functions. In section 4.3, we consider compositions of approximate maps, which are themselves approximate maps. In section 5, we show how the approximate map may optimally implement the abstract data type of the *approximate relation*. Finally, in later sections, we explore approximate rank-ordered search based on *first-order* random approximate maps.

2 Approximate maps

The concept of a *map* is dependent upon the concept of a *set*.

Definition 2.1. *A set is an unordered collection of distinct elements from a universe of elements.*

A countable set is a *finite set* or a *countably infinite set*. A *finite set* has a finite number of elements. For example,

$$S = \{1, 3, 5\}$$

is a finite set with three elements. A *countably infinite set* can be put in one-to-one correspondence with the set of natural numbers.

The cardinality of a set S is a measure of the number of elements in the set, denoted by

$$|S|. \tag{6}$$

The cardinality of a *finite set* is a non-negative integer and counts the number of elements in the set, e.g.,

$$|\{1, 3, 5\}| = 3.$$

Definition 2.2 (Cartesian product). *Let A and B be sets. The set $A \times B = \{(a, b) : a \in A \wedge b \in B\}$ is called the Cartesian product of the sets A and B .*

Definition 2.3 (Binary relation). *Let A and B be sets. By a relation $\mathcal{R}$ on A and B , we mean a subset of the Cartesian product $A \times B$. Since the Cartesian product is over two sets, we denote this relation a binary relation.*

Relations may be generalized to n -ary relations, which are subsets of the Cartesian product over n sets. We consider only *binary relations* known as *maps* as given by the following definition.

Definition 2.4. *A map $f: X \mapsto Y$ is a binary relation on $X \times Y$ with the constraint that the first element in the set of ordered pairs is unique.*

Such a map is also known as a *partial function*.

Consider discrete partial maps of type $\mathbf{X} \rightarrow \mathbf{Y}$. Any function f of this type may be *lifted* to the type $\mathbf{X} \mapsto \mathbf{m}(\mathbf{Y})$ where elements not in $\text{def}(f)$ map to a special value denoted by *nothing* that is not a member of $\mathbf{Y}$.

Given a map, by convention, in an ordered pair (x, y) , the first element x is denoted a *key* and the second element y is denoted the *value* associated with that (unique) key. The value y associated with key x may be denoted by

$$f(x). \quad (7)$$

The set of keys in the map is denoted by $\text{dom}(f) \subseteq \mathbf{X}$ and the set of values is denoted by $\text{codom}(f) \subseteq \mathbf{Y}$. Given these definitions, a map f is given by the set of ordered key-value pairs

$$f: \mathbf{X} \mapsto \mathbf{Y} = \left\{ (x, f(x)) \in \mathbf{X} \times \mathbf{Y} : x \in \text{dom}(f) \right\}. \quad (8)$$

A map $f: \mathbf{X} \mapsto \mathbf{Y}$ may represent *many-to-one* relationships, where many keys may be related to the same value. Consequently, denoting a particular key by its associated value is not generally possible.

A *table* is a convenient way to represent a map, where each key $x \in \text{dom}(f)$ is associated with $f(x) \in \text{codom}(f)$. The column for the keys consists of *unique* values from $\mathbf{X}$ and column for the values consists of (possibly repeated) values from $\mathbf{Y}$.

Example 1 Suppose we have a map $f: \{1, 3, 7\} \mapsto \{2, 7\}$ defined by the set of ordered key-value pairs

$$\{(3, 2), (1, 5), (7, 2)\}. \quad (\text{a})$$

We see that $\text{dom}(f) = \{3, 1, 7\}$, $\text{codom}(f) = \{2, 5\}$, $f(3) = 2$, $f(1) = 5$, and $f(7) = 2$. Notice that both 3 and 7 map to 2. Elements may be repeated in the 2^{nd} index of ordered pairs, but elements in the 1^{st} index must be unique.

Table 1 also depicts f where the keys column must consist of unique elements from $\mathbf{Z}$ and the values column must consist of (possibly repeated) elements from $\mathbf{Z}$.

Table 1: A table representing map $f: \mathbf{Z} \mapsto \mathbf{Z}$.

domain	range
3	2
1	5
7	2

2.1 Rate-distortion

Suppose we have the discrete metric space $(\mathbf{m}(\mathbf{Y}), d)$ where $d: \mathbf{m}(\mathbf{Y}) \times \mathbf{m}(\mathbf{Y}) \mapsto \mathbb{R}_{\geq 0}$. If we have a distribution X with a sample space $\mathbf{X}$ then the *expected* distance with respect to f and g is given by

$$\mathbb{E}_X[d(f, g)] = \sum_{x \in \mathbf{X}} p_X(x) d(f(x), g(x)). \quad (9)$$

If p_X is uniformly distributed, then $\mathbb{E}_X[d(f, g)]$ is the *average* distance $\bar{d}(f, g)$ over all values.¹

¹One way of defining a related discrete metric space $(\mathbf{m}(\mathbf{Y})^{\mathbf{X}}, d)$ where $d: (\mathbf{X} \mapsto \mathbf{m}(\mathbf{Y})) \times (\mathbf{X} \mapsto \mathbf{m}(\mathbf{Y})) \mapsto \mathbb{R}_{\geq 0}$ is given by letting $d(f, g) := |\mathbf{X}| \bar{d}(f, g)$.

Suppose we have an objective map $f: \mathbf{X} \mapsto \mathbf{m}(\mathbf{Y})$ and an approximation of f denoted by $f^\pm: \mathbf{X} \mapsto \mathbf{m}(\mathbf{Y})$. Then, we call d on the metric space $(\mathbf{m}(\mathbf{Y}), d)$ a *loss* function where the expected loss of the approximate function g of the objective function f is given by

$$\ell_X(g|f) = \mathbb{E}_X[d(f, g)]. \quad (10)$$

By the axioms of probability, if it is given that the random variable $X \in \mathcal{A}$ where $\mathcal{A}$ is a subset of sample space $\mathbf{X}$, then the conditional loss, denoted by $\ell_{X|\mathcal{A}}$, is given by

$$\ell_{X|\mathcal{A}}(g|f) = \mathbb{E}_{X|\mathcal{A}}[d(f, g)] \quad (11)$$

where $p_{X|\mathcal{A}}$ is normalized over $\mathcal{A}$.

Suppose the domain $\mathbf{X}$ may be partitioned into n disjoint sets $X_1, \dots, X_n$. The expected loss may be rewritten as

$$\ell_{X|\mathcal{K}}(g|f) = \sum_{i=1}^n \ell_{X|\mathcal{K} \cap X_i}(g|f). \quad (12)$$

Ideally, an approximation g of f is selected or chosen such that $\ell_{X|\mathcal{A}}(g|f)$ is minimized over some subset $\mathcal{A} \subseteq \mathbf{X}$ of interest. However, the expected loss may not obtain 0 due to *rate-distortion*, incomplete knowledge, noise, and memory or computing constraints.

3 Random approximate maps

The set of functions of type $\mathbf{X} \mapsto \mathbf{Y}$ has cardinality $|\mathbf{Y}|^{|\mathbf{X}|}$. We may also denote functions of this type by $\mathbf{Y}^{\mathbf{X}}$. Given a function $f: \mathbf{X} \mapsto \mathbf{Y}$, a randomly chosen function $g: \mathbf{X} \mapsto \mathbf{Y}$ equals f when applied to $x \in \mathbf{X}$ with probability $\frac{1}{|\mathbf{Y}|}$ and therefore $g = f$ with probability $|\mathbf{Y}|^{-|\mathbf{X}|}$.

We generalize this result and consider *random approximate maps*.

Definition 3.1. *The Iverson bracket, denoted by $\llbracket \cdot \rrbracket: \{\mathbf{true}, \mathbf{false}\} \mapsto \mathcal{B}$, maps a predicate to 1 if true and otherwise 0.*

For example, $\llbracket a < b \rrbracket = 1$ if $a < b$ is **true**.

Let $Y_x := \llbracket f(x) = f^\pm(x) \rrbracket$ be a Boolean random variable. The probability that for all x in $\mathbf{X}$, $Y_x = 1$ is given by

$$P[\cap_{x \in \mathbf{X}} Y_x = 1]. \quad (13)$$

If we assume independence,

$$P[\cap_{x \in \mathbf{X}} Y_x = 1] = \prod_{x \in \mathbf{X}} P[Y_x = 1]. \quad (14)$$

Since Y_x for each $x \in \mathbf{X}$ denote independent Boolean random variables, they are *Bernoulli* distributed.

Definition 3.2. *TODO: talk about independence. The order of a random approximate map corresponds to the minimum number of partitions in which every element of a given partition is identically and independently Bernoulli distributed random variables. We model $f: \mathbf{X} \mapsto \mathbf{m}(\mathbf{Y})$ by a random approximate function denoted by $f^\pm$. That is, $f^\pm$ is a random function.*

We denote a k -th order random approximate map of f by $f_k^\pm$ or, if we know and wish to emphasize the partitions, $f_{X_1, \dots, X_k}^\pm$ or the Bernoulli probabilities corresponding to each partition, $f_{p_1, \dots, p_k}^\pm$.

Since each partition consists of some number of Bernoulli distributed random variables, that means each partition has a *binomially distributed* number of 1's.

In the random approximate map model, we assume that $\mathbf{X}$ is uniformly distributed.

3.1 Approximate set-indicator function

We may view a set A with respect to its *indicator* function $\mathbb{1}_A: X \mapsto \mathcal{B}$.

The set indicator function partitions X into two parts, $P := \{x \in X \mid \mathbb{1}_A(x) = 1\}$ and $N := \{x \in X \mid \mathbb{1}_A(x) = 0\}$.

Definition 3.3. The false positive rate, denoted by $\hat{\varepsilon}$, is defined as the expectation $\ell_{X|N}(g|f)$ and the true negative rate, denoted by $\hat{\eta}$, is defined as the probability $1 - \hat{\varepsilon}$.

Definition 3.4. The false negative rate, denoted by $\hat{\omega}$, is defined as the expectation $\ell_{X|P}(g|f)$ and the true positive rate, denoted by $\hat{\tau}$, is defined as the probability $1 - \hat{\tau}$.

Remark. If P is a priori known, we can ignore elements not in the domain of definition of f and thus discard $\hat{\varepsilon}$. $\triangle$

By eq. (12) and definitions 3.3 and 3.4, the expected loss over the entire domain is given by

$$\ell_X(g|f) = \ell_{X|N}(g|f) p_X(N) + \ell_{X|P}(g|f) p_X(P) = \hat{\varepsilon}(1 - p_X(P)) + \hat{\omega} p_X(P). \quad (15)$$

The false negative rate is given by $\omega = \ell_{X|P}(f, f^\pm)$.

The false positive rate is given by $\varepsilon = \ell_{X|N}(f, f^\pm)$.

Each element in N is a false positive with probability ε . That is, it is a Bernoulli trial with $p = \varepsilon$. If a false positive occurs, then it randomly chooses a *representation* in the prefix-free codes for Y or an undefined code, which we map to **nothing**.

If the values in Y are uniquely represented, then it randomly (uniformly) chooses a value in Y .

It may hash to an undefined code. Two possibilities to consider:

1. The hash to undefined code that is not a prefix of any other code.
2. If the codes are variable-length, it may hash to a prefix of one or more codes.

If the values in Y are uniquely represented, then it randomly (uniformly) chooses a value in Y .

3.2 Probability model

TODO 3:

General first-order random approximate map model is given by the following definition.

Definition 3.5. A function $f_{A^+}: X \mapsto Y$, $A \subseteq X$, is a first-order random approximate map of $f: X \mapsto Y$ if the following conditions are satisfied:

$$f_{A^\pm}(x) := \begin{cases} Y^+ & x \notin A, \\ Y_- & x \in A. \end{cases} \quad (16)$$

where Y^+ is a random variable with a support that is a subset of Y (including being equal to Y).

If we have a first-order rate-distorted map, then Y^+ has a probability mass function that is given by

$$p_{Y^+}(y) = \sum_{b \in \text{encode}_Y(y)} 2^{-\ell(b)}, \quad (17)$$

where encode_Y is a prefix-free encoder for values in Y .

An optimal encoder may be found by computing the frequency of each element in Y when we apply f to every element in A or X^2 and generating a Huffman code for it.

However, since encoding the Huffman code may require a relatively large amount of memory, especially if the image of f is large, this approach may not be practical. Alternatively, there are many well-known algorithms to generate prefix codes for specific distributions, or families of distributions. Universal codes may be appropriate when the image of f is very large. If all else fails, there is always the uniform distribution.

Example 2 *The first-order positive random approximate set A_ε^+ is a first-order random approximate map of $\mathbb{1}_A: X \mapsto \mathcal{B}$ where, conditioning on A , $\sum_{b \in \text{encode}_{\mathcal{B}}(1)} 2^{-\ell(b)} = \varepsilon$ and $\sum_{b \in \text{encode}_{\mathcal{B}}(0)} 2^{-\ell(b)} = 1 - \varepsilon$.*

If there are constraints, such as the space of bijective functions $X \mapsto X$, then there are $|X|!$ such functions and the probability that, given a bijective function f , a randomly chosen bijective function g is equal to f with probability $\frac{1}{|X|!}$.

TODO -1:

The distance function $d(a, a) = 0$ and otherwise 1 combined with ℓ counts the proportion of values mapped correctly.

TODO 0:

The *zero-th* order approximation? Maybe we say that its uniformly distributed, i.e., just a random variable with a sample space Y .

What is first order then? Partition into k subsets, and elements in partition j are identically distributed.

What is second order? Partition into k subsets, and when we condition on a subset ...

TODO .5: Is 1st order actually 4th order model? Partions X into A and $\bar{A}$. Then, it partitions $\bar{A}$ into $\{y^*\}$ and $\bar{A} - \{y^*\}$ and it partitions A given $x \in A$ into $\{f(x)\}$ and $Y - \{f(x)\}$. That makes 4 partitions, each element in which has a different conditional probability.

We shouldn't overcomplicate though. Maybe just say these are random approximate maps, sometimes difficult to quantify distributions, but in some cases we can provide a well-understood model, e.g., the set-indicator function has a first-order model that's straight-forward. Even higher-order models are somewhat understandable. Boolean algebra types in general, including $\{0, 1\}$, all have this easily understood first-order model?

TODO 1:

Revised first-order model: $f_{A^\pm}: X \mapsto Y$ has the property that if $x \in A$ then $Y = f_{A^\pm}(x)$ realizes $f(x)$ with probability $1 - r$ and is uniformly distributed over sample space $Y - \{f(x)\}$ otherwise, i.e., realizes any value in $Y - \{f(x)\}$ with probability $r/(|Y| - 1)$.

For each $x \notin A$, $Y = f_{A^\pm}(x)$ is independently and identically distributed where it realizes some default value y^* with probability $1 - \varepsilon$ and is uniformly distributed over $Y - \{y^*\}$ otherwise.

TODO 1.5:

Don't necessarily mention rate distortion except in passing, and then talk about, say, SHM as generating errors due to rate distortion.

TODO 2:

Given a function $f: X \mapsto Y$, $f_{Q_\varepsilon^q}: X \mapsto Y_q$ approximately models $f \upharpoonright_Q: X \mapsto Y$ where elements not in $Q \subseteq X$ are mapped by $f \upharpoonright_{Q^\pm}$ to Y with probability ε and to some *default* value q with probability $1 - \varepsilon$.

²The latter may be chosen if we care about the approximation error over $X \setminus A$.

The *first-order random approximate set*, A_ε^ω , may be modeled by the approximate map of the characteristic function, $f_{A_\varepsilon^\omega}^0 : U \mapsto \mathcal{B}$.

If we wish to speak of the *type*, we may move the notation to the arrow, i.e., $X \xrightarrow[\omega]{\varepsilon} Y$.

The default value may be disjoint from Y , e.g., the sum type $Y + q$.

If we do not know the rates, or they are unimportant, we may instead denote it by $f_{Q^\pm} : X \mapsto Y$.

$f : X \mapsto Y$, $f_{Q^\omega} : X \mapsto Y$

We say that is of type $f : X \mapsto Y$, $f_{Q^\omega} : X \mapsto Y$

Additionally, elements in Q are not mapped by $f|_{Q^\pm}$.

TODO: revisit false negative and false positive rate. False positive rate should probably just be about what the approximate map is conditioned on, i.e., if $f : X \rightarrow Y$ and we build $\text{amap}(f, A)$ then we have $fA : A \rightarrow m(Y)$. If $fA(x) \neq \text{Nothing}$ when x not in A , then that is a false positive. Simplify this result. Now we can focus more on false negatives. A false negative is when $fA(x) \neq f(x)$ when x in A . In other words, we want to look at the restriction of the original function f to $A \rightarrow m(Y)$ (partial) and compare it to fA .

Now, for the approx boolean, $fA : \text{bool} \rightarrow \text{bool}$ (or $m(\text{bool})$), we typically let $A = 0,1$. So, in that case, there are no "negatives" and so no false positives, there is only a rate distortion on the positives. Only two positives, 0 and 1. If $fA(\text{false})$ maps to Nothing , that's a false negative... but shouldn't really happen in this case the function is so simple. Now we must also talk about the maybe monad though... because a $\text{approx}(\text{bool})$ is $0,1,\text{nothing}$ with some prob. distribution over the elements, $P[fA(\text{true}) = \text{nothing}]$ is false negative rate. $P[fA(\text{true}) \neq \text{nothing}]$ is 1-fnr. Rate distortion on positives that don't map to Nothing are a different matter. Now we're talking about

4 Higher-order random approximate maps

Typically, a data structure models a *first-order* random approximate map where the approximation error is due to *rate-distortion*. Then, through algebraic operations on first-order maps, such as composition, *higher-order* approximate maps are the natural result.

We discuss some of these higher-order models in this section.

4.1 Algebra of relations

4.2 Algebra of sets

The *set-complement*, *set-intersection*, and *set-union* operators have set-indicator functions defined respectively as

$$\mathbb{1}_{\bar{A}} := 1 - \mathbb{1}_A(x), \quad (18)$$

$$\mathbb{1}_{A \cap B}(x) := \mathbb{1}_A(x) \mathbb{1}_B(x), \quad (19)$$

$$\mathbb{1}_{A \cup B}(x) := \mathbb{1}_A(x) + \mathbb{1}_B(x) - \mathbb{1}_A(x) \mathbb{1}_B(x). \quad (20)$$

4.3 Composition

Composition of first-order approximate maps yields (potentially) a higher-order approximate map where elements in the negative set are not identically distributed.

The higher the order of the approximation, the greater the number of different distributions.

The positive set is more difficult to quantify since an algorithm may try to generate maps that yield minimal error.

If the ...

Compose should return a function that is the composition of a list of functions of arbitrary length. Each function is called on the return value of the function that follows. You can think of compose as moving right to left through its arguments.

Let $f_{\varepsilon_1}^{\omega_1} : X \mapsto Y$ be an approximate map with a false positive rate ε_1 and false negative rate ω_1 and let $g_{\varepsilon_2}^{\omega_2} : Y \mapsto Z$ be an approximate map with a false positive rate ε_2 and false negative rate ω_2 .

Then, the composition $g^{\pm} \circ f^{\pm} : X \mapsto Z$ is an approximate map.

Assumption 1. *When composing $f^{\pm}$ with $g^{\pm}$, if $x \in \text{dom}(f)$, then $f(x) \in \text{dom}(g)$. Otherwise, the composition is undefined.*

Let X denote a randomly chosen element from X . Under the assumptions of our model, what is the probability that X results in undefined behavior when used as the input in the composition

$$\begin{array}{c} f_{\varepsilon}^{\omega} \circ g_{\varepsilon}^{\omega} ? \\ + \\ - \mapsto \\ \varepsilon \\ - \rightarrow \\ \omega \\ \varepsilon \mapsto \\ \varepsilon_3 \\ \omega_2 \mathbf{A}^{\omega_2} \\ \varepsilon_1 \mathbf{A}^{\varepsilon_2} \\ \omega_3 \\ \varepsilon \\ \mathbf{A} \\ \omega \\ \varepsilon \mathbf{A} \times \varepsilon \mathbf{A} \\ + \\ \mathbf{A} \\ \pm \mathbf{A} \end{array}$$

Suppose $\omega_1 = \omega_2 = 0$. If it is given that $X \in \text{dom}(f)$, the probability of a false positive is zero. Otherwise, a false positive occurs if:

1. $X \in \text{dom}(f_{\varepsilon}^{\omega})$ and $f_{\varepsilon}^{\omega}(X)$ maps to a value $Y \in \text{dom}(g_{\varepsilon}^{\omega})$.

Suppose we have an approximate map where the keys are from the tuple of n elements given by the type

$$X = X_1 \times \cdots \times X_n \quad (21)$$

and the values are from the tuple of m elements given by the type

$$Y = Y_1 \times \cdots \times Y_m. \quad (22)$$

Example 3 *Let the keys be of the type*

$$X = \{1, 2\} \times \{a, b\} \quad (a)$$

and the values be of the type

$$Y = \{c, d\} \times \{1, 2\}. \quad (b)$$

Since these are small sets, for may enumerate the universe of keys and values. The universe of keys is given by

$$(1, a), (1, b), (2, a), (2, b) \quad (c)$$

and the universe of values is given by

$$(c, 1), (c, 2), (d, 1), (d, 2). \quad (d)$$

Then, the map $f: X \mapsto Y$ maps tuples of the form $(x_1, \dots, x_n), x_j \in X_j$ to values of $(y_1, \dots, y_m), y_j \in Y_j$.

Currying is a way of constructing functions that allows partial application of a function's arguments.

Example 4 Let the approximate map $f: Z \times Z \mapsto Q \times Z$ be defined by the rule

$$f(x_1, x_2) = \left(\frac{x_2}{2}, x_1 \right) \quad (a)$$

for $(x_1, x_2) \in \{(1, 1), (2, 2), (3, 4)\}$. The set of ordered pairs defining f is thus given by

$$\left\{ \left((1, 1), \left(\frac{1}{2}, 1 \right) \right), \left((2, 2), (1, 2) \right), \left((3, 4), \left(\frac{3}{2}, 4 \right) \right) \right\}. \quad (b)$$

Define composition of functions. Then, talk about how a composition of functions with approximate domains result in composition of functions with approximate domains. Talk about how the false positive rate increases. Relate this to an error measure?

One kind of composition is fuzzy hedge functions?

5 Predicate functions

5.1 Indicator functions (unary predicate functions)

The indicator function of a set A over a universal set U is denoted by $\mathbb{1}_A: X \mapsto \mathcal{B}$. A *first-order* random approximate set of A with a false positive rate ε and false negative rate ω is denoted by A_ε^ω , which may be modeled by a *first-order* random approximate map $\mathbb{1}_{A_\varepsilon^\omega}: X \mapsto \mathcal{B}$. More generally, functions of type $X \xrightarrow[\omega]{\varepsilon} \mathcal{B}$ model first-order random approximate subsets of the universal set X . This may also be indicated by R_ε^ω given that R may realize any element in 2^X .

5.2 Relations as n -ary predicates

A k -ary predicate function is a k -ary *relation*. For the k -ary predicate on k oblivious types, we may use either *approximate relations* or perfect relations. However, a perfect relation for oblivious types requires a bit length that is a function of the total possible number of k -relations on the types whereas the approximate relation on oblivious types only requires a bit length that is a function of the the number of relations in the exact relation. Except for very small types, the approximate relation is the only practical option.

TODO: mention how the oblivious relation can be also oblivious to the *arity* of the relation. Could be a unitary relation, could be a k -ary relation where k is arbitrary large. This is good for preventing random “searches” over tuples from effectively determining actual relations.

TODO: need to flesh out the oblivious relation (approximate relation) paper. Then, a k -ary predicate is a k -ary relation. It's not necessarily a function, it's only asking if a given k tuples are *related*. For instance, an ordered set answers questions like $a \leq b$ if (a, b) is in the binary relation.

A k -ary predicate satisfying the *regular function* concept, $f: T_1 \times \dots \times T_k \mapsto \text{Bool}$, maps k -tuples of type $(T_1, \dots, T_k)$ to **true** or **false**.

Higher-order may result from many sources.

Suppose we have two binary predicates $f: X \times Y \mapsto \mathcal{B}$ and $g: Y \times Z \mapsto \mathcal{B}$. To model a k -ary predicate, we might chain the tests, e.g., a 3-ary predicate is approximated by $w: X \times Y \times Z \mapsto \mathcal{B}$ with a definition

$$w(x, y, z) := f(x, y) \wedge g(y, z). \quad (23)$$

Theorem 5.1. *The false positive rate of a composed approximate predicate is bounded by the sum of the individual false positive rates.*

Theorem 5.2. *The false negative rate of a composed approximate predicate depends on the logical composition of the individual predicates.*

Theorems 5.1 and 5.2 is the *ideal probabilistic model* for k -ary predicates over *oblivious types*. If some subsets of $[T_1] \times \dots \times [T_k]$ are more likely to have false positives, and therefore some subsets are less likely to have false positives (and likewise for false negatives), then for instance, determining the probability that a particular tuple is a false positive or a true positive may be done with greater probability than given by our model assumptions.

5.2.1 Total orders

Less-than predicates

6 Discrete partial functions

6.1 0-ary functions (constants)

The special case of the 0-ary predicate is a *constant true* or *false*. This may be useful in some cases, notably *cipher* value types. To put it simply, if we have a constant function $f: 1 \mapsto X$, then $f(1) := y$ and $f^\pm(1)$ is equal to y with probability $1 - \omega$ and some other value with probability ω .

Basically, approximate constants. In *algebraic data types*, the approximate constant is fundamental. We may compose such things to generate other types, e.g., the Boolean type $\mathcal{B}$ is just $f^\pm(1) + g^\pm(1)$, yielding an approximate Boolean type.

7 A first-order rate-distorted map

We present a *theoretical* data type, denoted the *entropy map* that have several key properties:

1. Perfect model of the *first-order* random approximate map.
2. Bit-length of in-place encoding is a random variable L that is expected to obtain the theoretical lower-bound.
3. In-place encoding has maximum conditional entropy (when conditioning on a few parameters like expected false positive rate).

Note that we know a priori what the distribution of the random number of trials N needed to produce a result that satisfies the rate-distortion constraints where a parameterized Goulumb code is expected to compress the representation. However, this representation is not *in-place* and is thus only useful as a *serialization* format.

We consider sets in which the universe of elements is given by the *countably infinite* set of all bit strings.

Definition 7.1. The countably infinite set of all bit strings is denoted by

$$\mathcal{B}^* = \{b: b \in \{0, 1\}^*\}, \quad (24)$$

where $*$ denotes all non-negative integers.

A finite subset of $\mathcal{B}^*$ is given by the following definition.

Definition 7.2. The finite set of all bit strings of length n is denoted by

$$\mathcal{B}^n = \{b: b \in \{0, 1\}^n\} \quad (25)$$

with a cardinality given by

$$|\mathcal{B}^n| = 2^n. \quad (26)$$

A hash function is related to countable sets $\mathcal{B}^*$ and $\mathcal{B}^n$ and is given by the following definition.

Definition 7.3. A hash function of type $\mathcal{B}^* \mapsto \mathcal{B}^n$ maps (hashes) bit strings of arbitrary-length to bit strings of fixed-length n .

The random oracle is given by the following definition.

Definition 7.4. A random oracle $h: \mathcal{B}^* \mapsto \mathcal{B}^\infty$ is a theoretical hash function whose output is uniformly distributed over its codomain.

The bit length of objects is given by the following definition.

Definition 7.5. The bit length of an object x is denoted by

$$\ell(x). \quad (27)$$

For example, the bit length of an object $x \in \mathcal{B}^n$ is $\ell(x) = n$.

The set of rationals in $[0, 1]$ is denoted by $\mathcal{P}$ and the set of power-of-two probabilities is denoted by

$$\mathcal{R} := \{2^{-k} \in \mathbb{Q} \mid k \in \mathbb{Z}_{\geq 0}\} \subset \mathcal{P}. \quad (28)$$

Suppose we a prefix-free encoder-decoder for values of type Y denoted respectively by $\text{encode}_Y: Y \mapsto \mathcal{B}^*$ and $\text{decode}_Y: \mathcal{B}^* \mapsto \mathfrak{m}(Y)$.

If we are given a (possibly infinite) sequence of bits $b_1 b_2 \dots b_k b_{k+1} \dots$ and $b_1 b_2 \dots b_k$ is a (prefix-free) code for some y in Y , then $\text{decode}_Y(b_1 b_2 \dots b_k b_{k+1} \dots)$ maps to y . Inversely, $\text{encode}_Y(y)$ maps to $b_1 b_2 \dots b_k$.

The singular hash map models a *first-order* random approximate map where the approximation error is due to *rate-distortion*. *Rate-distortion* is a result of two different parameter choices:

1. Distortion due to *insufficient* bits.
2. Distortion due to accepting any false negative rate less than some lower-bound.
3. Distortion due to specifying a false positive rate, which causes elements not in the set to be mapped to be mapped to some uncertain value that is a function of the prefix-free encoder-decoder. If codes assume a uniform distribution, then randomly choose a Y .

In the singular hash map, item 1 is due to either a larger *false positive rate* $\varepsilon = 2^{-k}$ by increasing k or restricting the search to a maximum finite number trials (and choosing the trial that resulted in the minimum loss according to some distance function).

In either case, the end result is an expected smaller encoding length, but in the first case we are directly motivated by a rate limit in the form of an upper-bound on the bit length and in the second case we are satisfied with greater approximation error but impose no upper-bound.

In the metric space (X, d) , if d is defined as $d(a, a) = 1$ and otherwise $d(a, b) = 0$, $a \neq b$, we get the

The probability of success given a rate-distortion ω on positives and ε on negatives is given by

$$p(\varepsilon, \omega) = \binom{m}{(1-\omega)m} \varepsilon^{m(1-\omega)} (1-\varepsilon)^{m\omega}. \quad (29)$$

Each trial is geometrically distributed, $T \sim \text{GEO}(p(\varepsilon, \omega))$.

The expected number of trials is $E[T] = \frac{1}{p(\varepsilon, \omega)}$. The expected code size is $E[\log_2 T]$, which is non-linear and problematic but may be approximated with

$$-H_b(\omega) - \omega \log_2(1-\varepsilon) - (1-\omega) \log_2 \varepsilon \quad (30)$$

bits per element where $H_b(p) := (1-p) \log_2(1-p) + p \log_2 p$ is the binary entropy function.

This result is not the result of computing $E[\log_2 T]$, nor the expectation of its Taylor approximation. It is simply $\log_2 E[T]$.

If ω and ε are near 0, then this simplifies to $\omega \log_2 \omega -$

If we let $\omega := 0$, the above simplifies to $\log_2 \varepsilon$, which is the lower-bound for *first-order* random positive approximate sets over countably infinite universes with a false positive rate ε .

NB —

Talk about NB distribution.

If we are satisfied with false negatives, we are no longer requiring all m to succeed.

Negative binomial: given n trials, what is the probability k succeeds? If $k = m$, we are back to the previous solution with **fnr** = 0.

The expected false negative rate is just not due to one trial, but an expectation

$$\sum_{k=0}^m P[T = k] \quad (31)$$

If T realizes **fnr** that means after n trials, the best was **fnr**.

—

7.1 Maybe monad

Suppose the encoder-decoder has the pair $\text{encode}_X: X \mapsto 2^{\mathcal{B}^*}$ and $\text{decode}_X: \mathcal{B}^* \rightarrow X$.

We consider a specialization of the algorithm that, given a function $f: X \mapsto Y$ generates rate-distorted maps $f_\varepsilon^\omega: X \mapsto \mathfrak{m}(Y)$. The encoder-decoder only applies to elements of X which test as positive, at which point the elements are mapped to values as previously.

In other words, and assuming no rate-distortion, it maps values in the domain of definition of $f|_A$ to *nothing* with probability ω and otherwise maps it correctly and it maps values not in its domain of definition to *nothing* with probability ε and value y with probability

$$p_Y(y) := \sum_{z \in \text{encode}_Y(y)} 2^{-\ell(z)}. \quad (32)$$

Figure 1: *Singular Hash Map*

The function $\text{generate_hash_seeds}: (\mathsf{X} \mapsto \mathsf{m}(\mathsf{Y})) \times 2^{\mathsf{X}} \times \mathcal{R} \mapsto 2^{\mathcal{B}^* \times \mathcal{P}}$ is defined as

$$\text{generate_hash_seeds}(\mathsf{f}, \mathsf{A}, \varepsilon) := \mathsf{T} \quad (33)$$

where

$$\begin{aligned} m_a &:= |\mathsf{A}|, \\ k &:= -\log_2 \varepsilon, \\ h(x|n, k) &:= \text{tr}(h(x' \# n'), k), \\ \text{match}(x|n, k) &:= h(x|n, k), \\ \mathsf{H} &:= \{\text{match}(x|n, k) \in \mathcal{B}^k \mid x \in \mathsf{X}\}. \end{aligned} \quad (34)$$

Algorithm 1: Implementation of `construct_singular_hash_map` over a universal set X

input : A is a subset of a universal set X . ε is the false positive rate. t is the maximum number of trials.

output : A^ε , a rate-distorted approximate set of A .

```

1 function construct_singular_hash_map( $\mathsf{A}$ ,  $\varepsilon$ ,  $t$ )
2    $\mathsf{A}_{\mathcal{B}^*} := \{\text{encode}_{\mathsf{X}}(a) \in \mathcal{B}^* \mid a \in \mathsf{A}\}$ 
3   for  $n \leftarrow 0$  to  $t$  do
4     for  $j \leftarrow 1$  to  $2^n$  do
5        $\text{found} \leftarrow \text{true}$ 
6        $b_n \leftarrow$  a bit string randomly drawn from  $\mathcal{B}^n$  without replacement
7        $h_k \leftarrow \text{nothing}$ 
8       for  $x \in \mathsf{A}_{\mathcal{B}^*}$  do
9         if  $h_k = \text{nothing}$  then
10           $h_k \leftarrow \text{tr}(h(x \# b_n), k)$ 
11          else if  $h_k \neq \text{tr}(h(x \# b_n), k)$  then
12             $\text{found} \leftarrow \text{false}$ 
13       if  $\text{found}$  then
14         return  $\langle h_k, b_n \rangle$ 
```

In Algorithm 1 on Line 10, the value v_i has a *self-terminating* encoding. Thus, we need only check that the bit string that encodes v_i is equal to a hash of the key (concatenated with another 1 and another bit string b).

The space required for the *singular hash map* found by Algorithm 1 is of the order of the length n of the bit string b . Therefore, for space efficiency, the algorithm exhaustively searches for a bit string in the order of increasing size n .

7.2 Theoretical analysis

The *singular hash set* proves that the keys in $\mathsf{f}^\pm$ are an *approximate set* of the keys in f .

Theorem 7.1. *The probability that a particular bit string $b \in \mathcal{B}^*$ successfully codes the singular hash map is given by*

$$p(m, \varepsilon, \mu) = \frac{\varepsilon^{m-1}}{2^{m\mu}}. \quad (35)$$

Algorithm 2: Implementation of `construct_singular_hash_map` over a universal set $\mathcal{X}$

input : A is a subset of a universal set $\mathcal{X}$. ε is the false positive rate. t is the maximum number of trials.

output : A^ε , a rate-distorted approximate set of A , .

```
1 function construct_singular_hash_map( $A, \varepsilon, t$ )
2    $A_{\mathcal{B}^*} := \{ \text{encode}_{\mathcal{X}}(a) \in \mathcal{B}^* \mid a \in A \}$ 
3   for  $i \leftarrow 0$  to  $t$  do
4     found  $\leftarrow$  true
5      $b \leftarrow$  a bit string randomly drawn from  $\mathcal{B}^{\leq t}$  without replacement
6      $h_k \leftarrow$  nothing
7     for  $x \in A_{\mathcal{B}^*}$  do
8       if  $h_k = \text{nothing}$  then
9          $h_k \leftarrow \text{tr}(h(x \# b), k)$ 
10      else if  $h_k \neq \text{tr}(h(x \# b), k)$  then
11        found  $\leftarrow$  false
12    if found then
13      return  $\langle h_k, b_n \rangle$ 
```

Algorithm 3: Implementation of `construct_singular_hash_map` over a universal set $\mathcal{X}$

input : A is a subset of a universal set $\mathcal{X}$. ε is the false positive rate. t is the maximum number of trials.

output : A^ε , a rate-distorted approximate set of A , .

```
1 function construct_singular_hash_map( $A, \varepsilon, t$ )
2    $A_{\mathcal{B}^*} := \{ \text{encode}_{\mathcal{X}}(a) \in \mathcal{B}^* \mid a \in A \}$ 
3   for  $n \leftarrow 0$  to  $t$  do
4      $H \leftarrow \emptyset$ 
5     for  $b \in A_{\mathcal{B}^*}$  do
6        $H \leftarrow H \cup \{ \text{tr}(h(b \# n'), k) \}$ 
```

Proof. In algorithm 1, for a particular bit string b_n , the joint probability that every key in f collides and for each key k in f , the concatenation of 1 and k has a hash in which the first $\ell(v)$ bits collide with mapped value v is given by the following reasoning.

Suppose we have a set $f := \{(k_1, v_1), \dots, (k_m, v_m)\}$ and k_1 hashes to $y = h(k_1) \bmod t$, where $h: \mathcal{B}^* \mapsto \mathcal{B}^N$ is a random hash function that uniformly distributes over its domain of 2^N possibilities and $N < \max \ell(v_1), \dots, \ell(v_m)$.

The probability that $h(1 \# k_1) \bmod \ell(v_1) = v_1$ is given by

$$\frac{1}{2^{\ell(v_1)}}. \quad (\text{a})$$

Since y is a particular element in $\mathcal{B}^k$, the probability that $k_j, j \neq 1$, hashes to y is given by

$$\frac{1}{2^k}. \quad (\text{b})$$

The probability that $1 \# k_j, j \neq 1$ hashes to v_j is given by

$$\frac{1}{2^{\ell(v_j)}}. \quad (\text{c})$$

Since h is a random hash function, every one of these probabilities are independent, and thus the joint probability is just the product given by

$$p = \frac{1}{2^{k(m-1)}} \prod_{j=1}^m \frac{1}{2^{\ell(v_j)}} \quad (\text{d})$$

$$= \varepsilon^{m-1} \frac{1}{2^{\sum_{j=1}^m \ell(v_j)}}. \quad (\text{e})$$

The average bit length per key is given by

$$\mu = \frac{1}{m} \sum_{j=1}^m \ell(v_j) \quad (\text{f})$$

and thus the probability p may be parameterized as

$$p = \frac{\varepsilon^{m-1}}{2^{m\mu}}. \quad (\text{g})$$

□

Definition 7.6. In the singular hash map construction algorithm, the n -th trial uniquely maps to a bit string of length $m = \lfloor \log_2 n \rfloor$, establishing a correspondence between trial number and bit string length.

Theorem 7.2. Given false positive and false negative domain error rates given by ε and ω respectively, and a mean bit length μ of encodings of elements in the image of $f^\pm$, the expected bit length of the Singular Hash Map asymptotically obtains a space complexity with a lower-bound given by

$$-(1 - \omega) \log_2 \varepsilon + (1 - \omega) \mu \text{ bits/element}, \quad (\text{36})$$

which occurs when the encodings have uniformly distributed bit lengths, and an upper-bound given by

$$-(1 - \omega) \log_2 \varepsilon + \mu \text{ bits/element}, \quad (\text{37})$$

which occurs when the encodings have degenerate bit lengths. The greater the entropy of the bit lengths, the smaller the expected bit length given a mean of μ .

Proof. The space required for the singular hash set found by Algorithm 1 is of the order of the length n of the bit string b_n . Therefore, for space efficiency, the algorithm exhaustively searches for a bit string in the order of increasing size n .

We are interested in the first case when a success occurs, which is a geometric distribution with probability of success p as given by

$$Q \sim \text{GEO} \left(p = \frac{\varepsilon^{m-1}}{2^{m\mu}} \right). \quad (\text{a})$$

The expected number of trials for the geometric distribution is given by

$$E[Q] = \frac{2^{m\mu}}{\varepsilon^{m-1}}. \quad (\text{b})$$

By Definition 7.6, the n -th trial uniquely maps to a bit string of length $m = \lfloor \log_2 n \rfloor$. Thus, the expected bit length is given approximately by

$$E[\log_2 Q] = \log_2 \frac{2^{m\mu}}{\varepsilon^{m-1}} \text{ bits}. \quad (\text{c})$$

Dividing by m and simplifying results in

$$-\frac{m-1}{m} \log_2 \varepsilon + \mu \text{ bits/element}. \quad (\text{d})$$

Asymptotically, as $m \rightarrow \infty$, the bits per element goes to

$$-\log_2 \varepsilon + \mu. \quad (\text{e})$$

□

The smaller the bit lengths generated by the encoder, the smaller the *Singular Hash Map*. However, optimal encoders *reveal* information about the values in the *Singular Hash Map*, which goes against the principle of the *oblivious function*. For instance, if each input x mapped to a list of numbers, then an optimal encoder will generate codes that minimize the length of the list, say a Huffman code. However, by analyzing the Huffman codes produced by the encoder, we may infer the distribution of the lists. Thus, for the sake of maintaining an *oblivious* state, the encoder should generate codes as though the values were *uniformly* distributed.

7.3 Entropy

References

- [1] Alexander Towell. The approximate relation abstract data type. 2018. URL ?
- [2] Alexander Towell. The approximate set abstract data type. 2018. URL ?